

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки/специальность
09.03.01 Информатика и вычислительная техника

направленность (профиль)/специализация
«Технологии разработки программного обеспечения»

Выпускная квалификационная работа

Разработка приложения для администрирования клиентской базы данных
медицинской организации

Обучающегося 4 курса
очной формы обучения
Угарина Никиты Александровича

Руководитель выпускной квалификационной
работы:
Старший преподаватель
Аксютин Павел Александрович

Рецензент:

Ученая степень (*при наличии*), ученое звание
(*при наличии*), должность
Ф. И. О. (*указывается в именительном
падеже*)

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ	2
ВВЕДЕНИЕ.....	3
ГЛАВА 1: ПЕРЕНОС И ДОРАБОТКА БАЗЫ ДАННЫХ ИЗ ACCESS.....	5
1.1 Ознакомление с требованиями для создания приложения	5
1.2 Ознакомление с существующим приложением	6
1.3 Анализ методологий разработки и миграции базы данных	9
1.4 Перенос базы данных из MS Access в MS SQL.....	11
1.5 Подведение итогов теоретической части	16
ГЛАВА 2: РАЗРАБОТКА WPF ПРИЛОЖЕНИЯ.....	17
2.1 Базовое ознакомление с приложением.....	17
2.2 Создание проекта и подключение к базе данных.....	18
2.3 Создание окна авторизации	22
2.4 Создание главного меню	26
2.5 Картотека.....	28
2.6 Журнал.....	30
2.7 Новая карточка	33
2.8 Новый прием.....	34
2.9 Вспомогательное окно	38
2.10 Создание отчетов	40
2.11 Сборка приложения	43
ЗАКЛЮЧЕНИЕ	45
СПИСОК ЛИТЕРАТУРЫ	46

ВВЕДЕНИЕ

В современных организациях, где имеется постоянный поток клиентов, существует необходимость хранить их данные и работать с ними. В школах это списки учеников и их успеваемость, в поликлиниках это данные о пациентах и их больничная карта и так можно перечислять ещё очень долго. Записывать всю информацию на бумаге не эффективно, ведь в таком случае крайне тяжело с ней работать в сжатые сроки. Ведь в таком случае любую статистику придётся высчитывать вручную, а сколько может занять поиск определённого человека в картотеке по его инициалам или, к примеру, по его определённым характеристикам сложно даже представить. Поэтому в наше время появляется необходимость отцифровывать информацию. Ведь если тот же самый учёт пациентов будет происходить на компьютере и в одной структурированной базе данных, то работа с этими данными будет гораздо проще и быстрее, особенно если этих пациентов будут тысячи и даже десятки тысяч в год.

В Петроградском районе Санкт-Петербурга существует государственное бюджетное учреждение центр психолого-педагогической, медицинской и социальной помощи «Здоровье», далее — ЦППМСП «Здоровье». Основной его задачей является психолого-педагогическая помощь детям. Для этого в организации ведётся учёт всех пациентов с помощью личных бумажных карт, а также имеется небольшая база данных в Microsoft Access 2013, в которой дублируются данные из карточек пациентов. В связи с ограниченным функционалом, устаревшим программным обеспечением, не стабильной работой и желанием перейти на собственное приложение, было принято решение о создании собственного программного обеспечения для администрирования базы данных, с улучшенным функционалом, возможностями и скоростью работы.

Актуальность определена заказом руководства ЦППМСП «Здоровье» Петроградского района Санкт-Петербурга. Данный проект оптимизирует процесс хранения и обработки информации о клиентах данной организации.

Цель работы: разработать приложение для записи, хранения и обработки информации о пациентах ЦППМСП «Здоровье».

Для достижения поставленной цели необходимо решить следующие задачи:

- Ознакомиться с существующей базой данных;
- провести анализ существующих способов разработки собственного программного продукта;
- перенести базу данных из Microsoft Access в выбранную систему управления базами данных (СУБД);
- создать приложение для администрирования данной базы данных

Работа состоит из введения, первой главы, которая включает в себя анализ существующей базы данных, анализ различных способов разработки программных продуктов для администрирования базой данных, выбор наиболее подходящего, в данной ситуации, метода создания приложения и переноса базы данных из Microsoft Access в другую систему управления базами данных, и второй главы, которая описывает практическую реализацию задач, поставленных ранее, заключения и списка литературы.

ГЛАВА 1: ПЕРЕНОС И ДОРАБОТКА БАЗЫ ДАННЫХ ИЗ ACCESS

1.1 Ознакомление с требованиями для создания приложения

Для корректного понимания функционала приложения требуется ознакомиться с самой организацией и сутью её работы. В России существует Государственное бюджетное учреждение центр психолого-педагогической, медицинской и социальной помощи "Здоровье" Петроградского района Санкт-Петербурга. Данная организация занимается оказанием психолого-педагогической помощи детям от 0 до 18 лет, их родителям, а также образовательным учреждениям Петроградского района по вопросам обучения и воспитания детей с проблемами школьной и социальной адаптации. Ежедневно в организацию обращаются люди с детьми, некоторые люди приходят впервые, а кто-то не в первый раз. Для каждого посетителя (ребенка) создаётся его личная карточка (карточка пациента). В ней указываются все необходимые данные, начиная с фамилии, имени, отчества, пола, возраста, а заканчивая информацией о родителях и данными о местах обучения, кто направил и так далее. Все данные записываются на физическую карту.

При посещении организации карточка пациента передаётся от регистратуры специалисту, где он записывает в неё информацию о приёме, такую как: дату, вид приёма, был ли приём первичный или повторный, причину обращения, а также заключения по результатам приёма. Постепенно карточка наполняется записями различных специалистов, а поиск каких-либо определённых занятий или подсчёт статистики становится едва возможным, так как ежегодно в организации происходит более трёх тысяч обращений и приёмов.

Для решения данной проблемы от меня требовалось создать приложение для администрирования всей картотеки учреждения. Создание собственного приложения позволило бы вести более быстрый учет посетителей, быстрее обрабатывать и искать информацию по ним и вести статистику.

1.2 Ознакомление с существующим приложением

У организации имелось приложение для администрирования базы данных посетителей, использующее Microsoft Access 2007. Это система управления базами данных (СУБД), входящая в пакет Microsoft Office. Это реляционная СУБД, база данных которой состоит из взаимосвязанных таблиц. Пользователь, работающий в данном приложении, фактически работает одновременно с таблицами и с панелью управления.

У данного приложения существует серьёзный минус, из-за которого было решено уйти в создание собственного приложения, а именно максимальный размер базы данных — 2 Gb. Как только база разрастается, скорость операций вставки и выборки падает катастрофически, из-за чего наблюдаются постоянные зависания и ошибки в её работе. Также у приложения имеется ограниченный функционал, который необходимо дополнить.

Внутри приложения я смог наблюдать данную схему данных (см. рис. 1)

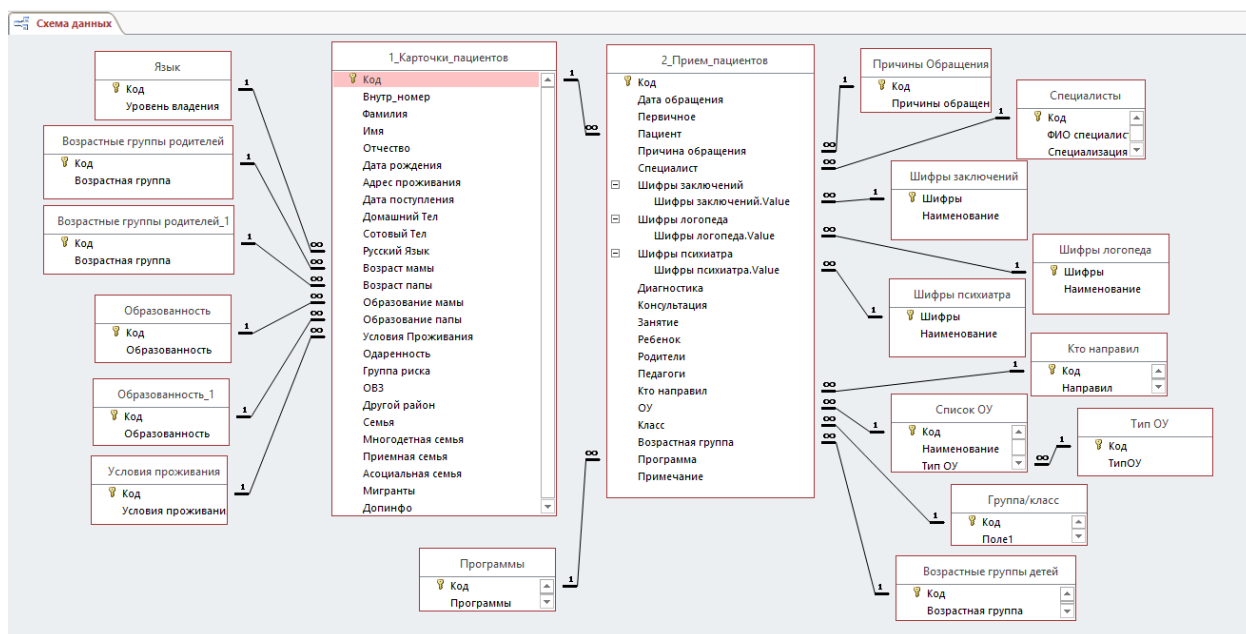


Рисунок 1 — Схема данных Access

Внимательно изучив интерфейс пользователя, мною были выделены основные окна управления базой данных, а именно «Карточка» и «Приём».

Скриншот интерфейса «Карточка» (Patient Card) в медицинской базе данных. Вверху — панель заголовка с вкладками «Старт» и «Карточка», заголовком «Карточка №» и полем «Дата поступления» со значением 12.03.2026.

Основная форма содержит следующие поля:

- Личные данные: Фамилия, Имя, Отчество, Дата рождения, Возраст, Адрес проживания (с опцией «Другой район»).
- Контакты: Тел. Домашний, Тел. Сотовый.
- Язык и условия: Русский Язык (выпадающий список), Условия Проживания (выпадающий список).
- Одаренность: ☐ ОВЗ ☐.
- Родители: Таблица с колонками «Родители:», «Возраст» и «Образование» для Мама и Папа.
- Семья: Полная (выпадающий список), Асоциальная семья, Многодетная семья, Приемная семья, Мигранты, Группа риска.
- Дополнительная информация: Большое текстовое поле.

В нижней части формы — таблица с заголовками: Первичное, Причина обращения, Специалист, Шифры заключений, Шифры, Шиф, Диагнос, Консультат, Занят. В таблице есть одна строка с данными: * (галочка), (пустое поле), (пустое поле), (пустое поле), (пустое поле), (пустое поле), (галочка), (галочка).

Внизу — панель управления записями: «Запись: 1 из 1», «Нет фильтра», «Поиск», и кнопки «Сохранить в картотеке» и «Вернуться в главное меню».

Рисунок 2 — Карточка пациента

Окно «Карточка» позволяет заполнить данные о новом пациенте: ввести личную информацию и дополнительную. Хочу обратить внимание на минус данного окна — порядковый номер карточки. Его приходилось вбивать вручную, опираясь на знания и память последнего номера существующей карты, что порождало множество трудностей, а также потенциальные ошибки в нумерации карт. Чтобы избежать данную проблему необходимо ввести функцию автоматического установления нового номера карточки, следующего от максимального существующего.

Второе по значимости окно «Приём», в него вписываются данные о приеме посетителя у специалиста.

Рисунок 3 — Окно приема пациента

В этом окне требуется выбрать ребенка из выпадающего списка, есть возможность поиска по фамилии, но из-за функциональных ограничений Microsoft Access, а также очень большой базы данных пациентов, в момент поиска нужного человека приложение постоянно зависает и даже закрывается. Также выпадающими списками выбираются остальные блоки. Имеются окна с выбором шифров заключений, по результатам приема у различных специалистов. В нижней части окна две кнопки «Сохранить в журнале» и «Вернуться в главное меню». Само главное меню имеет простой интерфейс, с кнопками, для создания новой карточки пациента и нового приема пациента.

Исходя из информации на рисунке 1, можно сделать вывод, что основными таблицами в данной БД являются «1_Карточки_пациентов» и «2_прием_Пациентов». Также можно сделать вывод, что данная база данных реляционная и денормализованная. Для разработки прикладного решения для конкретного учреждения будет использоваться подход, учитывающий особенности эксплуатации и вид существующей базы данных. Ряд положений нормальных форм будет сознательно нарушен в пользу повышения производительности и упрощения интерфейса для пользователя.

1.3 Анализ методологий разработки и миграции базы данных

При выполнении дипломной работы ключевым этапом являлся выбор архитектуры и инструментов для реализации программного продукта. Изначальная система была реализована в Microsoft Access, что накладывало определенные ограничения. В рамках данной работы было принято решение о переходе от файловой архитектуры к клиент-серверной с использованием MS SQL Server в качестве СУБД и Windows Presentation Foundation (WPF) на платформе C# для разработки приложения. Чтобы объяснить мой выбор, необходимо немного углубиться в терминологию.

SQL Server Management Studio (SSMS) — это интегрированная среда от Microsoft, предназначенная для доступа, настройки, управления и администрирования всех компонентов SQL Server. Объединяет графический интерфейс, редактор SQL-запросов и инструменты администрирования баз данных. SSMS было разработано как компьютерное приложение, тесно интегрированное с экосистемой ОС Windows, включая компоненты .NET Framework. Главное преимущество данной среды в том, что в ней существует официальный инструмент для миграции базы данных с Access в SSMS. А называется он — SSMA (SQL Server Migration Assistant for Access — это бесплатный официальный инструмент от Microsoft, предназначенный для автоматизации миграции баз данных из Microsoft Access).

.NET Framework — это программная платформа, созданная корпорацией Microsoft, предназначенная для запуска и разработки приложений под Windows. Она выступает промежуточным звеном между операционной системой и работающими в ней программами. Основная задача .NET Framework — предоставить среду для запуска и разработки приложений, упростить создание и поддержку приложений.

.NET и SSMS являются продуктами одной экосистемы Microsoft, что обеспечивает их бесшовную интеграцию на всех уровнях взаимодействия. Эта синергия проявляется в следующих аспектах:

Аспект интеграции	Описание
Унифицированная модель подключения	ADO.NET — это стандартный механизм доступа к данным в .NET, который изначально разрабатывался для работы с SQL Server. В SSMS используются те же принципы подключения, что обеспечивает единообразие настройки соединений.
Общие протоколы аутентификации	Обе платформы поддерживают единые механизмы безопасности: Windows Authentication, SQL Server Authentication и современные методы с использованием Microsoft Entra ID (ранее Azure Active Directory) .

Подводя итоги, можно сделать вывод, что разработка с использованием .NET и SSMS одновременно позволит мне быстрее и проще создавать собственное приложение, так как SSMS предоставляет встроенные средства миграции из Microsoft Access через SQL Server Migration Assistant (SSMA), что позволяет автоматизировать и упростить перенос существующей структуры данных и, главное, сохранить целостность информации, а платформа .NET обеспечивает удобные механизмы доступа к этим данным без необходимости написания сложного кода для преобразования типов и обработки соединений.

Для разработки визуальной части приложения была выбрана Windows Presentation Foundation (WPF) — система для построения клиентских приложений Windows с визуально привлекательными возможностями взаимодействия с пользователем. К тому же эта графическая подсистема

находится в составе .NET Framework, точно также как и SSMS. Также стоит ознакомиться с особенностями данной технологии:

- В основе лежит векторная система визуализации, не зависящая от разрешения устройства вывода, что крайне удобно для разработки, при планирующихся дальнейших условиях использования на разных компьютерах.
- В данной системе используется язык разметки XAML (eXtensible Application Markup Language) для декларативного описания интерфейса, основанный на XML. XAML используется для описания пользовательского интерфейса (UI) в приложениях, созданных с помощью технологий Microsoft.
- Логика приложений, использующих XAML, основывается на языке программирования C#.

Из информации, предоставленной выше, можно сделать логичное заключение о том, какие будут использоваться технологии разработки. Осталось определиться со средой разработки приложения. Так как разработка будет происходить на WPF с использованием C# и SSMS, то необходимо выбрать среду разработки, поддерживающую данные методы. Идеально для данных условий подходит Visual Studio — интегрированная среда разработки также от Microsoft, которая поддерживает создание приложений на C# с использованием WPF. В ней существует встроенный XAML конструктор пользовательского интерфейса и редактор кода. Также Visual Studio имеет возможность подключения к базам данных SQL Server.

1.4 Перенос базы данных из MS Access в MS SQL

Для успешного переноса базы данных будет использоваться, упомянутый ранее, инструмент, разработанный компанией Microsoft — SQL Server Migration Assistant for Access (SSMA), который я упоминал ранее. Запустив программу, после нажатия кнопки создания нового проекта,

появляется интерактивное окно, в котором требуется ввести первоначальные настройки миграции базы данных. Разработка будет происходить на SQL Server версии 2022 года. Также надо указать название проекта и путь для его хранения.

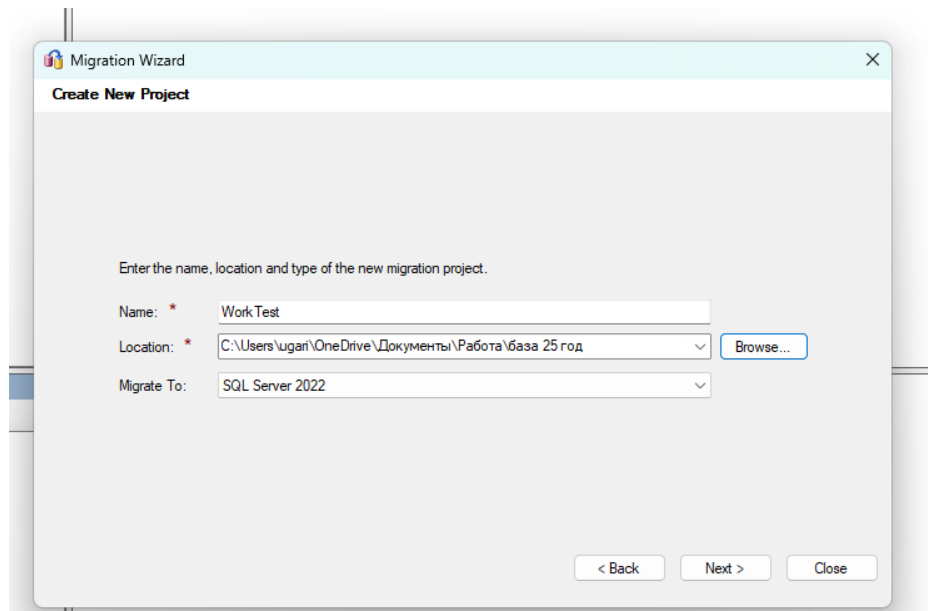


Рисунок 4 — Окно настроек проекта 1

В следующем окне требуется нажать кнопку «Add database» и выбрать файл существующей базы данных, которую необходимо перенести. Затем в новом окне выбрать базовую настройку. В пункте «Server name» необходимо указать «.\SQLEXPRESS» — это имя экземпляра SQL Server. Обратная косая черта и точка перед именем указывает, что это именованный экземпляр, установленный локально на компьютере. SQLEXPRESS — стандартное имя экземпляра для SQL Server Express. Также необходимо указать метод аутентификации, о котором я говорил ранее, «Windows Authentication» и поставить галочки в пунктах, отвечающих за доверие сертификату сервера и за обязательное шифрование данных, для обеспечения большей безопасности. В названии базы данных указывается название созданного проекта. Пункт «Server port» оставляется по умолчанию.

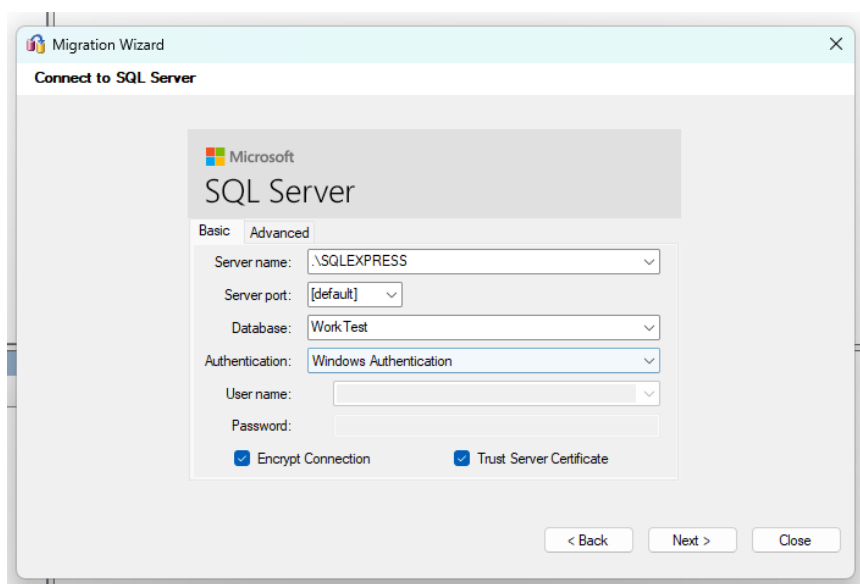


Рисунок 5 — Окно настроек проекта 2

Следующим шагом требуется просто запустить перенос базы данных, нажав на кнопку «Create database». Подождав небольшое количество времени, программа предоставит нам файл под названием «Database». Внутри будет находиться папка с таким же названием, как и исходный файл MS Access. Внутри этой папки надо выбрать «Tables», там будут находиться перенесенные программой таблицы.

Теперь, если зайти в SQL Server Management Studio, появляется возможность открыть перенесенный проект. В меню «Connect to Server» необходимо указать те же параметры, что и при создании проекта в SQL Server Migration Assistant for Access, а также проставить галочку в пункте, отвечающем за доверие сертификату сервера. Важное примечание: иногда в программе бывает сбой, из-за которого программа не может найти перенесенную базу данных по пути «.\SQLEXPRESS», поэтому придется вместо точки указать имя компьютера, в моем случае необходимо ввести строку «UGES\SQLEXPRESS».

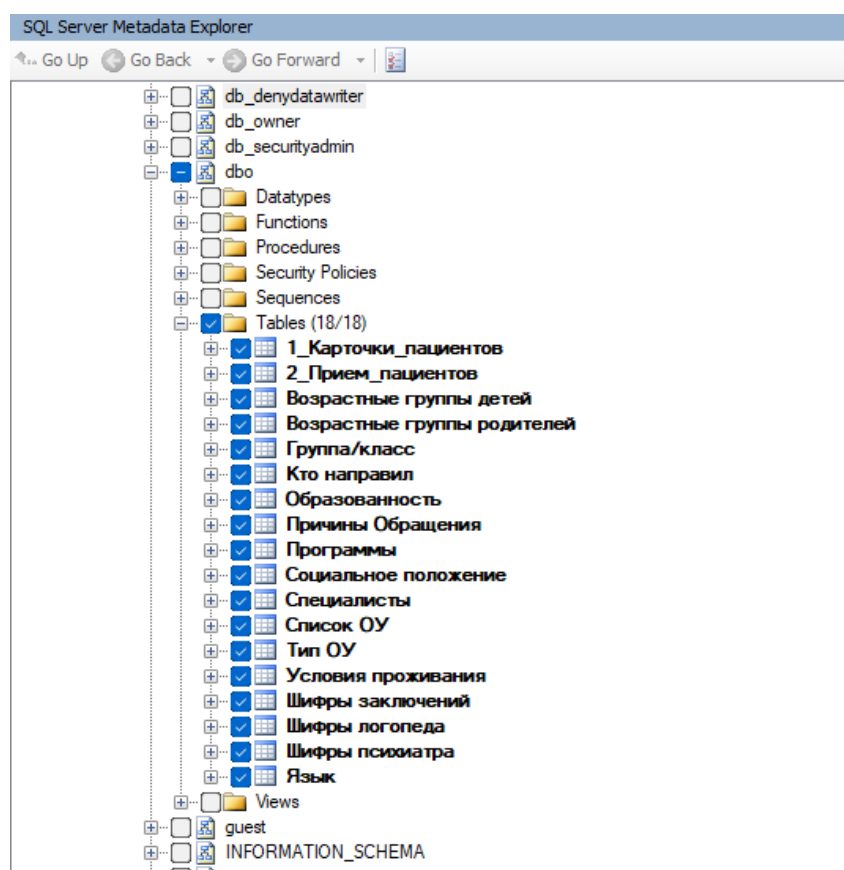


Рисунок 6 — Результат переноса базы данных

Следующий и крайне важный шаг — изменение имен таблиц в базе данных. В моем случае, в программе MS Access, таблицы имели названия, состоящие из нескольких слов, идущих через пробел. При переносе таблиц названия перенеслись корректно, но из-за того, что управление таблицами будет происходить через код на языке программирования C#, а не через внутренний интерфейс программы, как было в Microsoft Access, для корректного восприятия таблиц необходимо заменить все пробелы на символ нижнего подчеркивания.

Следующим шагом, будут созданы таблицы «Roles» и «Users», для обеспечения дальнейшего входа в приложение по логину и паролю. Это повысит безопасность данных и не позволит третьим лицам пользоваться приложением, так как весь функционал будет находиться под паролем. Для создания таблиц, в меню навигации, необходимо нажать правой кнопкой мыши по пункту «Tables», затем «New» и выбрать «Add table». В таблице

пользователей будет две колонны «id_role» и «name_role», а в таблице ролей необходимо указать колонны, отвечающие за номер пользователя, имя для входа, пароль и номер роли. Затем необходимо связать эти таблицы, а конкретнее номер роли от таблицы ролей к таблице пользователей. Такая связь позволит использовать, к примеру, для роли менеджера несколько учетных записей, что в свою очередь позволит разным сотрудникам работать под своей личной учетной записью, а руководству отслеживать их действия. Чтобы сделать данную связь, надо в панели навигации нажать на пункт «Database Diagrams» и добавить новую диаграмму, выбрав в открывшемся меню все таблицы. Затем в таблице пользователей выбрать «id_role», нажать правой кнопкой мыши и выбрать пункт «Set Primary Key». Точно также необходимо сделать в таблице ролей. Данный пункт ставит ограничение для выбранного столбца, который уникально идентифицирует каждую строку в таблице. Это ограничение обеспечивает целостность данных, так как не позволяет вставлять строки с дублирующимися значениями. Затем достаточно взять за появившийся ключ и протянуть его из таблицы ролей в пользователи.

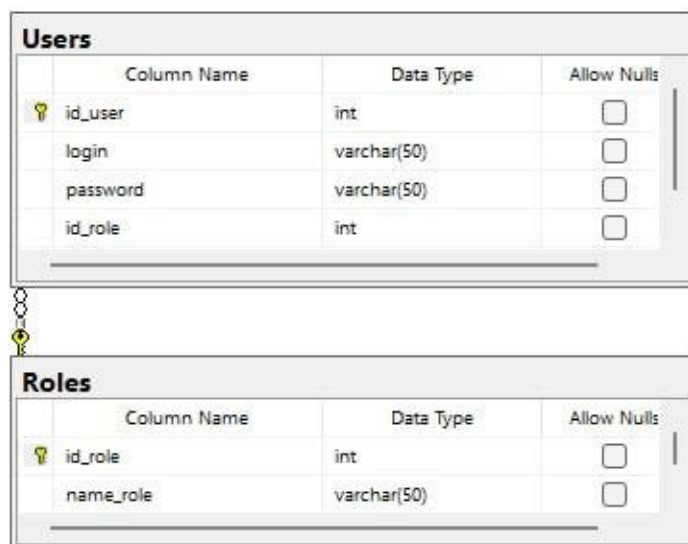


Рисунок 7 — Таблицы пользователей и ролей

Дополнительно было необходимо отредактировать названия столбцов в старых таблицах, в том числе, таким образом, чтобы не было пропусков между

слов. Все пропуски заменялись на символ нижнего подчеркивания — «_». Также по требованию руководителя организации было проведено усовершенствование таблиц - добавление новых столбцов, для увеличения функционала. Когда таблица была полностью сделана и отредактирована, под требования организации, ее необходимо перенести в среду разработки. Более подробно о данном процессе будет рассказано в практической части выпускной квалификационной работы.

1.5 Подведение итогов теоретической части

В теоретической части были озвучены требования к созданию приложения для управления базой данных. Были разобраны термины: клиент-серверная архитектура, реляционная база данных, миграция данных, а также такие программные продукты и технологии, как Microsoft Access, SQL Server Management Studio (SSMS), SQL Server Migration Assistant (SSMA), .NET Framework, Windows Presentation Foundation (WPF), XAML, C# и Visual Studio. Был проведён детальный анализ существующей базы данных учреждения ЦППМСП «Здоровье», выявлены её недостатки: ограниченные возможности Microsoft Access, низкая скорость работы при большом объёме данных, нестабильность, устаревший интерфейс и трудности в использовании. На основе требований заказчика обоснован выбор технологического стека: переход на MS SQL Server в качестве СУБД и WPF на платформе .NET для разработки клиентского приложения. Описан процесс миграции базы данных из Access в SSMS с использованием инструмента SSMA, включая настройку подключения, корректировку имён таблиц и столбцов, а также создание дополнительных таблиц «Roles» и «Users» для разграничения прав доступа. Таким образом, теоретическая часть заложила фундамент для практической реализации программного продукта, обеспечивающего автоматизацию учёта пациентов, повышение скорости и безопасности работы с данными.

ГЛАВА 2: РАЗРАБОТКА WPF ПРИЛОЖЕНИЯ

2.1 Базовое ознакомление с приложением

Перед разработкой приложения обозначу все требования, которые были озвучены со стороны руководства ЦППМСП «Здоровье», а конкретно наличие таких блоков как:

- меню авторизации,
- главного меню после авторизации,
- окна «Картотека пациентов»,
- окна «Карточка пациента», с возможностью создания новой карточки,
- окна с возможностью создания нового приема пациента, опираясь на его карточку,
- журнала приема с функцией поиска,
- возможности создания отчетов в формате PDF.

Остальные и более подробные требования и функции по каждому пункту отдельно, будут мною описаны в детальном рассмотрении каждого окна управления. Графический интерфейс реализован на WPF, о котором писалось в теоретической части. Для каждого экранного компонента создано отдельное окно:

- MainWindow — авторизация;
- MainInterface — главное меню после входа;
- Cart — картотека пациентов (поиск и список);
- NewCard и NewCard2 — создание/редактирование карточки пациента и открытие существующей карточки;
- NewJoin — создание нового приема пациента;
- Journal — журнал приемов (просмотр с фильтрацией);
- отчеты — генерируются классами Report (не как отдельные окна, а как PDF-выход).

Навигация между окнами была реализована через открытие окон методом «Show()» и закрытие с помощью метода «Close()», в зависимости от сценария поведения пользователя.

Проект компилируется под .NET Framework 4.7.2. Для доступа к данным используется Entity Framework 6, причем модель данных сгенерирована по EDMX файлу: Entities/Model1.edmx. Сгенерированный контекст: Entities/testjobEntities7: DbContext (цифра 7 просто указывает на то, что это была моя 7 попытка обновления и нового подключения базы данных в проект).

2.2 Создание проекта и подключение к базе данных

Для начала написания приложения необходимо создать новый проект в Visual Studio 2022. Выбрав пункт «Создание проекта» необходимо выбрать шаблон для разработки. Требуется выбрать шаблон «Приложение WPF (.NET Framework)» с поддержкой языка C#, XAML и ОС Windows, так как именно с его помощью можно сделать приложение с интерфейсом, используя XAML. Затем достаточно дать проекту имя и указать его расположение, в следующих пунктах ничего менять не нужно.

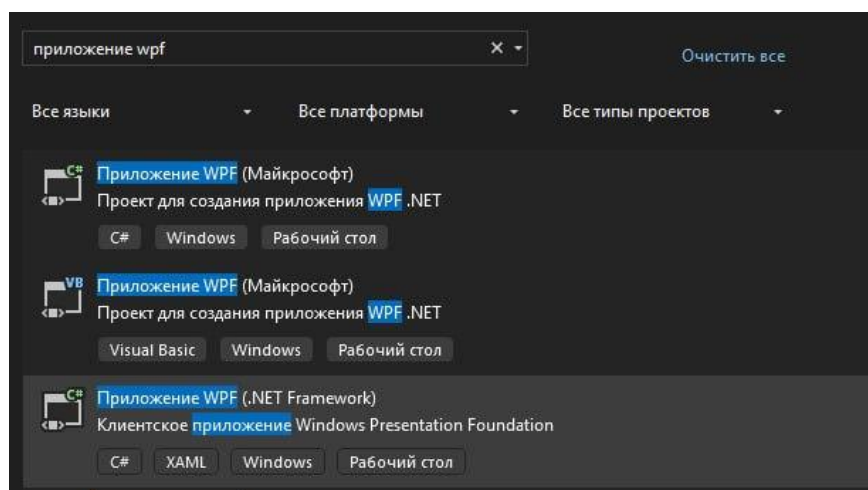


Рисунок 8 — Создание проекта

Оказавшись в главном меню разработки приложения, можно приступить к подключению, созданной ранее, базы данных. Первым делом для этого необходимо установить пакет «EntityFramework» версии 6 и выше (в моем

случае это будет версия 6.2.0). Без этого не соберётся код контекста DbContext — это ключевой класс в Entity Framework, который представляет собой соединение между приложением и базой данных. Данный класс управляет взаимодействием с данными, включая запросы, сохранение, отслеживание изменений и управление подключением к базе данных. Для установки пакета достаточно нажать ПКМ по проекту в обозревателе решений, затем выбрать пункт «Управление пакетами NuGet», а затем, в открывшемся окне, найти «Entity Framework» и установить его.

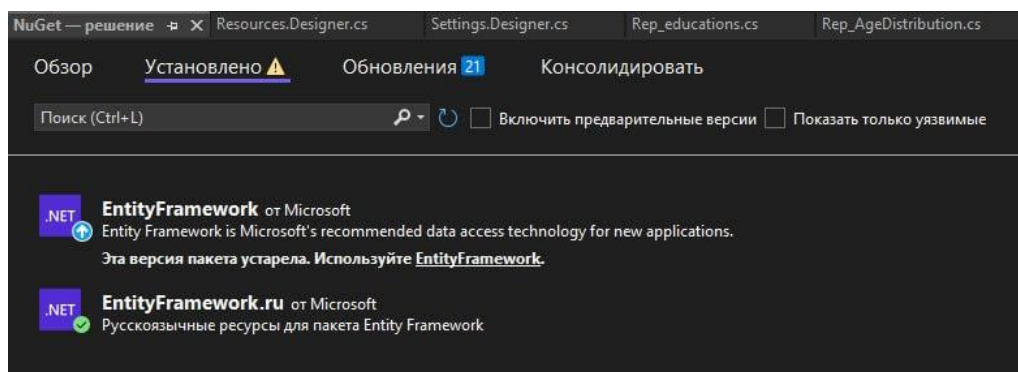


Рисунок 9 — Установка Entity Framework

Следующий и, на мой взгляд, самый важный этап — подключения базы данных. Для этого необходимо перейти в обозреватель решений, в котором происходит основная навигация по содержанию проекта. В обозревателе решений, нажатием ПКМ, открыть меню действий. Для подключения базы данных в проект требуется выбрать пункт «Добавить», а внутри его пункт «Класс». Для программного взаимодействия с базой данных в коде будет использоваться библиотека «ADO.NET». Пример некоторых классов, которые могут применяться:

- SqlConnection — используется для установления соединения с базой данных, которая хранится на MS SQL-сервере.
- SqlCommand — применяется для формирования SQL-запроса или вызова хранимой процедуры.

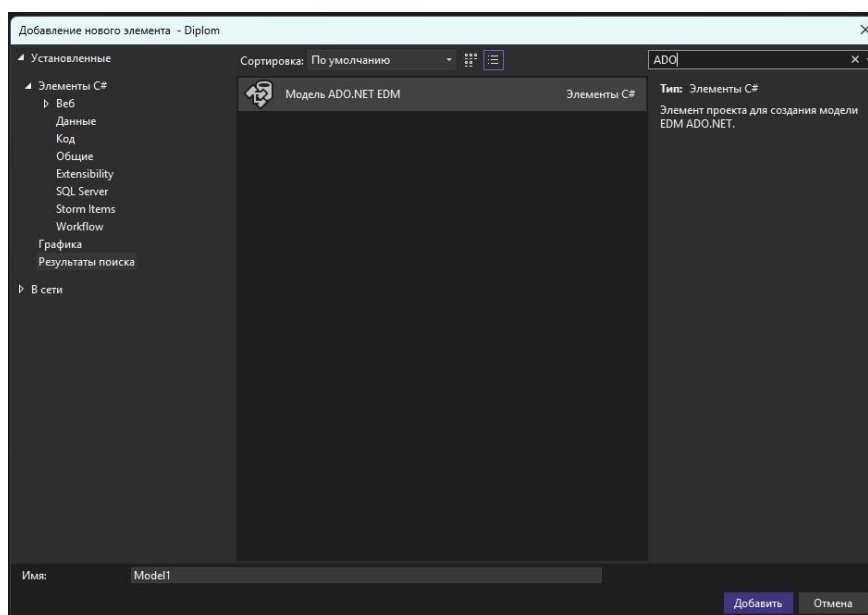


Рисунок 10 — Добавление ADO.NET

При добавлении «ADO.NET» потребуется указать имя модели, в моем случае «Model1». При указании пути ее размещения была указана папка «Entities». Предварительно ее также надо будет создать в обозревателе решений. Затем в мастере настройки подключения необходимо выбрать пункт «EF Designer from database» (модель из существующей БД) и настроить подключение к серверу. Так как сервер располагается локально, на том же компьютере, на котором и ведется разработка, то путь к серверу нужно указать «UGES\SQLEXPRESS». В данном случае «uges» это имя сервера (моего компьютера), на котором располагается база данных организации. Также обязательно надо указать название базы данных и поставить галочки в пунктах, отвечающих за доверие сертификату сервера и т.д. Когда программа попросит выбрать используемые таблицы, необходимо будет выбрать все и завершить настройку.

В качестве результата получаем, что в папке «Entities» появляется файл Model1.edmx, который состоит из сгенерированных Model1.Designer.cs, Model1.Context.cs, шаблонов Model1.tt и Model1.Context.tt, внутри которых находятся классы сущностей (Users.cs, Roles.cs и т.д.).

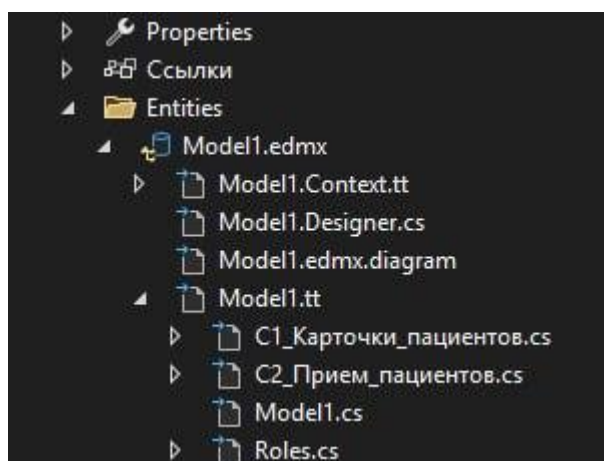


Рисунок 11 — Результат подключения БД

В результате появятся изменения в файле «App.config». Этот файл обновляется сам автоматически, при корректном подключении к базе данных. Самый важный блок внутри данного файла это <connectionStrings>. Он отвечает за подключение к базе данных. В данном блоке содержатся настройки, которые были указаны в мастере подключения базы данных.

Также чтобы не создавать контекст «DbContext» для всех окон, в файле «App.xaml.cs» потребовалось добавить строчку — «public static Entities.testjobEntities context { get; } = new Entities.testjobEntities();». В данной строке «Entities.testjobEntities» — это класс, сгенерированный Entity Framework, который представляет собой контекст базы данных. Он наследуется от «DbContext», а «context» — экземпляр этого контекста, через который появляется возможность:

- Получать доступ к таблицам БД
- Отслеживать изменения объектов
- Сохранять изменения в БД

Теперь проект полностью настроен, в него были добавлены компоненты, отвечающие за подключение к базе данных. Также была подключена база данных и обновлен файл «App.xaml.cs».

2.3 Создание окна авторизации

Пожалуй, самый важный этап в разработке любого приложения для работы с большим количеством конфиденциальной информации — это обеспечение безопасности и сохранности данных. При использовании данных через приложение они будут автоматически шифроваться, данная функция автоматически установлена в SQL Server 2022. В таком случае остается одна и самая важная часть для обеспечения полной безопасности данных — создание меню авторизации пользователя. Данное меню позволит ограничить доступ к приложению третьим лицам.

Ранее мною были созданы таблицы «Users» и «Roles». Для тестовой проверки работы приложения была создана учетная запись администратора, которая в последствии, после внедрения, будет заменена на учетную запись по требованию руководства организации.

Для создания первого окна требуется нажать в обозревателе решений правой кнопкой мыши и выбрать пункт «Окно WPF» тем самым создастся пустое окно, в котором будет проходить дальнейшая работа. После проделанных действий в Visual Studio можно наблюдать созданное белое окно, ниже, в конструкторе, xaml код данного окна и это же окно в обозревателе решений.

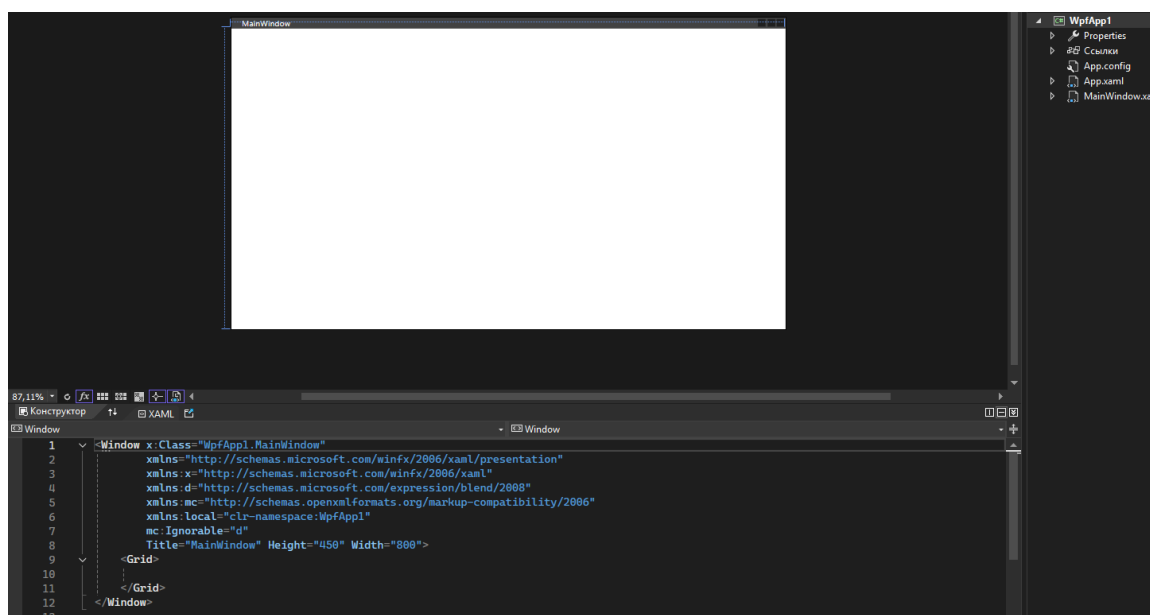


Рисунок 12 — Первое окно

Большая часть работы по модернизации и внедрению функционала окон будет происходить непосредственно с их кодом. Для начала будет произведена замена и настройка блока «Window». Этот блок непосредственно отвечает за отображение самой страницы. В нем автоматически указывается к какому классу и в каком проекте это окно принадлежит, указывается его название, можно добавить изображение. Это основной пункт, с которыми будут проводиться манипуляции. Заменяя модуль «Title» на такую строчку «Title="База - Здоровье" Height="1080" Width="1920" Background="AliceBlue" WindowState="Maximized"» устанавливаются стандартные параметры окна, его название изменяется, а цвет заднего фона заменяется на голубой. Затем появляется возможность создать папку для хранения изображений, к примеру логотипа приложения и окон, загрузить туда изображение, а затем добавить модуль «Icon="Images/logo.jpg"» в любом месте внутри блока «Window».

Далее производится настройка сетки разметки (Grid). В данном окне создается структура из трех строк: верхняя высотой 50 пикселей под шапку,

средняя указывается символом «*», это означает, что данная строка занимает все оставшееся пространство, и нижняя также высотой 50 пикселей, в текущей версии не используется, но оставлена для будущего функционала. Данный подход обеспечивает адаптивность интерфейса при изменении размеров окна. Верхняя панель содержит текстовый блок с названием учреждения «ЦППМСП Здоровье». Также настроены отступы, шрифт и цвет. Основная область реализует форму авторизации с вертикальным выравниванием в верхнем положении. В данной области последовательно размещаются:

- Текстовое поле «Логин»
- Поле ввода логина «TextBox»
- Текстовое поле «Пароль» с аналогичным оформлением
- Поле ввода пароля «PasswordBox» — важное отличие от TextBox, оно не отображает вводимые символы в открытом виде, обеспечивая безопасность
- Кнопка «Войти» с обработчиком события «Click»

Все элементы имеют фиксированные отступы от верхнего края, а белый фон полей ввода улучшает читаемость.

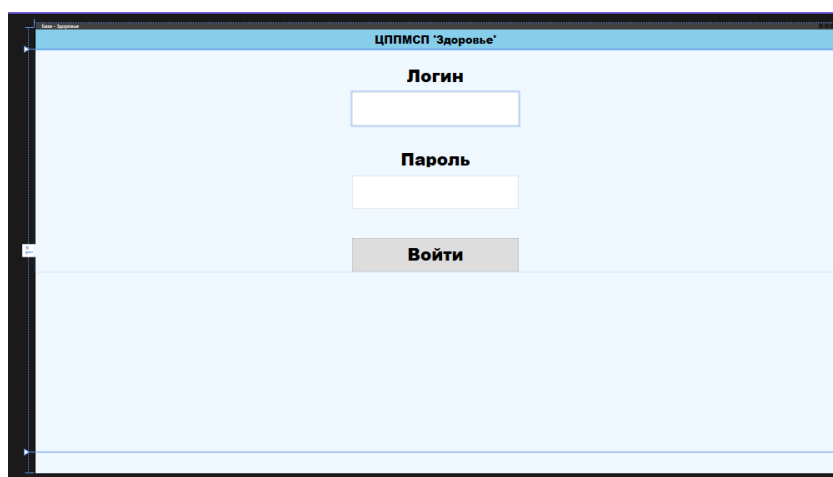


Рисунок 13 — Меню авторизации

Теперь требуется настроить C# код данного окна, для корректной работы с базой данных. Первое и самое важное - при запуске окна должен выполняться пробное обращение к таблице «Users», для поиска ролей. Если база данных недоступна, то будет возникать исключение, используя блок «catch». Пользователю выводится сообщение «База данных не подключена», а приложение закрывается. Это предотвращает работу системы без доступа к данным. Также добавляется учет событий клавиатуры, для полей «Login» и «Password» добавляются обработчики «LoginOrPassword_KeyDown». Это позволяет пользователю нажать клавишу «Enter», не перемещаясь к кнопке мышью, для вызова процедуры авторизации. Основной метод авторизации «LoginBtn_Click» выполняет следующие операции:

1. Поиск пользователя в базе данных: «App.context.Users.FirstOrDefault(p => p.login == Login.Text && p.password == Password.Password)».
2. Проверка успешной авторизации: если «currentUser» не равен нулю, это означает что пользователь найден, а система проверяет его роль через поле «id_role».
3. Обработка ошибок авторизации: если «currentUser» не существует (равен «null»), выводится сообщение «Неверный логин или пароль».

```
//нажатие клавиши ENTER
private void LoginOrPassword_KeyDown(object sender, KeyEventArgs e)
{
    if (e.Key == Key.Enter)
    {
        LoginBtn_Click(LoginBtn, new RoutedEventArgs());
    }
}
```

Рисунок 14 — Фрагмент кода в MainWindow

2.4 Создание главного меню

Следующий шаг — создание страницы главного меню. Требуется создать меню навигации в котором пользователь мог бы открыть окно создания нового приема пациента, в случае отсутствия пациента в базе данных, окно создания новой карточки пациента, окно просмотра журнала приема пациентов, а также окно просмотра картотеки. Аналогичным способом, как и с меню авторизации, создается окно «MainInterface». После успешной аутентификации пользователя с ролью «Администратор» («id_role» == 1) происходит закрытие окна авторизации и отображение главного меню. Окно было разделено также, как и предыдущее окно, на три строки, а также на три столбца, в автоматически подстраиваемой шириной. Первый столбец для кнопок перехода, описанных ранее, а последующие два, для создания кнопок отчетов, о которых будет описано в дальнейшем.

Центральная левая часть, а вернее вторая строка (Grid.Row="1"), нулевая колонка (Grid.Column="0") содержат четыре командные кнопки, реализующие переходы к основным функциональным модулям системы, с фиксированными отступами сверху.

Название кнопки	Содержимое	Отступ сверху	Назначение
Cart	Картотека	20 px	Переход к модулю просмотра и управления картотекой пациентов
Journal	Журнал приема	140 px	Переход к модулю просмотра истории приемов

NewCart	Новая карточка	260 рх	Переход к модулю создания новой карточки пациента
NewJoin	Новый прием	380 рх	Переход к модулю регистрации нового приема

Для корректного оформления фрагментов кода, отвечающих за переход на окна при нажатии кнопок, требуется создать эти окна. Для этого создам окна, отвечающие за журнал, картотеку, новую карточку и новый прием пациента. Они будут называться «Journal», «Cart», «NewCard» и «NewJoin» соответственно. Окна будут открываться с помощью встроенной функции «Show()», а также одновременно будет закрываться существующее окно, тем самым имитируя не просто открытие окна, а плавный переход, кроме картотеки и журнала, так как такое требование было у руководства организации.

```
// Открывает окно картотеки и закрывает текущее.
private void Cart_Click(object sender, RoutedEventArgs e)
{
    Cart cart = new Cart();
    cart.Show();
    //this.Close();
}
```

Рисунок 15 — Фрагмент кода, открывающего картотеку.

Для того, чтобы данный код срабатывал при нажатии кнопки, требуется добавить в саму кнопку обработчик нажатия, аналогично кнопке перехода в меню авторизации: «<Button x:Name="Cart" Click="Cart_Click" Width="400" Height="80" Content="Картотека" .../>». «Click="Cart_Click"» — это и есть обработчик, вызывающий функцию «Cart_Click», при нажатии на соответствующую кнопку.

2.5 Картотека

Открыв, созданную ранее, картотеку, применяются изменения в заголовке окна, аналогично другим окнам (заголовок, иконка и т.д.). Картотека нужна для отображения карточек всех пациентов из базы данных. Она будет представлять из себя таблицу реализованную элементом «ListView» с именем «CartLview». Для отображения данных в виде колонок используется «GridView» со следующими колонками:

Заголовок	Привязка к БД	Особенности
№	Внутр_номер	«TextBlock»
Фамилия	Фамилия	Прямая привязка
Имя	Имя	Прямая привязка
Отчество	Отчество	Прямая привязка
ДР	Дата_рождения	Форматирование dd.ММ.уууу
Возраст	Дата_рождения	Используется конвертер Age
Адрес	Адрес_проживания	Прямая привязка
Друг-рай	Другой_район	Используется конвертер Parser
Домашний тел.	Домашний_Тел	Прямая привязка

Столбец «№» отвечающий за номер карточки имеет отличный формат от других столбцов, так как в дальнейшем, после создания карточки пациента, планируется добавить возможность нажатия на номер карточки для перехода на подробное меню с описанием карточки данного пациента.

Для более приятного визуального отображения даты рождения используется форматирование — «StringFormat='{0:dd.MM.yyyy}'». Также для корректного отображения возраста потребовалось создать конвертер «Age», который будет высчитывать возраст пациента считая разницу между сегодняшней датой и датой его рождения.

```
if (value is DateTime dateValue)
{
    // Получаем текущую дату
    DateTime now = DateTime.Now;

    // Вычисляем разницу в годах
    int yearsDifference = now.Year - dateValue.Year;

    // Проверяем, был ли день и месяц в текущем году раньше, чем в исходной дате
    if (now.Month < dateValue.Month || (now.Month == dateValue.Month && now.Day < dateValue.Day))
    {
        yearsDifference--;
    }

    // Вычисляем разницу в месяцах
    int monthsDifference = now.Month - dateValue.Month + (yearsDifference * 12);

    // Если текущий месяц меньше, чем месяц в dateValue, уменьшаем на 1
    if (now.Day < dateValue.Day)
    {
        monthsDifference--;
    }

    // Получаем только полное количество месяцев, чтобы отобразить
    int totalMonths = monthsDifference % 12;

    // Формируем результат
    return $"{yearsDifference} г {totalMonths} мес"; // Возвращаем строку с количеством лет и месяцев
}

return "Недоступно"; // Возвращаем сообщение, если дата недоступна
```

Рисунок 16 — Логика работы конвертера «Age»

В картотеке используется два основных метода: загрузка данных (LoadPatientData()) и поиск и фильтрация (Search_TextChanged). Метод загрузки данных проверяет контекст базы данных — если «App.context» не инициализирован, выводится сообщение об ошибке, также проверяет доступность таблицы C1_Карточки_пациентов, формирует запрос с сортировкой по фамилии и имени, применяет AsNoTracking() — это ключевая оптимизация, так как Entity Framework не отслеживает изменения объектов, что снижает потребление памяти и увеличивает скорость загрузки, выполняет запрос и сохраняет результат в «_allPatientData» и устанавливает «ItemsSource» таблицы.

Метод содержит обработку исключений с выводом детального сообщения об ошибке для облегчения отладки.

Второй метод выполняет следующие функции: фильтрация «input-методу», выполняющаяся при каждом изменении текста в поле поиска. Алгоритм работает по такому принципу:

1. Проверка наличия загруженных данных.
2. Получение поискового запроса.
3. Если запрос пустой — отображается полный список.
4. Иначе происходит фильтрация с проверкой на «null» по следующим полям: фамилия, имя, отчество, адрес проживания, домашний и сотовый телефон, внутренний номер карточки.

Фильтрация выполняется в памяти, что обеспечивает мгновенный отклик интерфейса при вводе каждого символа. В дальнейшем будет добавлен метод отвечающий за открытие карточки пациента (CardNumber_MouseLeftButtonDown).

```
// Фильтруем данные в памяти
var filtered = _allPatientData
    .Where(p =>
        (p.Фамилия != null && p.Фамилия.ToLower().Contains(searchText.ToLower())) ||
        (p.Имя != null && p.Имя.ToLower().Contains(searchText.ToLower())) ||
        (p.Отчество != null && p.Отчество.ToLower().Contains(searchText.ToLower())) ||
        (p.Адрес_проживания != null && p.Адрес_проживания.ToLower().Contains(searchText.ToLower())) ||
        (p.Домашний_Тел != null && p.Домашний_Тел.Contains(searchText)) ||
        (p.Сотовый_Тел != null && p.Сотовый_Тел.Contains(searchText)) ||
        (p.Внутр_номер.ToString().Contains(searchText)) ||
        (p.Внутр_номер.ToString().StartsWith(searchText))
    )
    .ToList();
```

Рисунок 17 — Фрагмент кода — фильтрация данных

Итоговый результат окна журнала можно наблюдать во приложении №1.

2.6 Журнал

Следующим ключевым модулем данного приложения является окно - журнал обращений «Journal», предназначенное для просмотра истории посещений пациентов с возможностью фильтрации, детализации записей и быстрого перехода к полной карточке пациента. В отличие от окна картотеки «Cart», окно журнала работает с более сложными связями между таблицами

(порядок: приём, пациент, специалист, учреждение, причина обращения) и потенциально большим объёмом данных, так как у одного пациента может быть большое множество приемов.

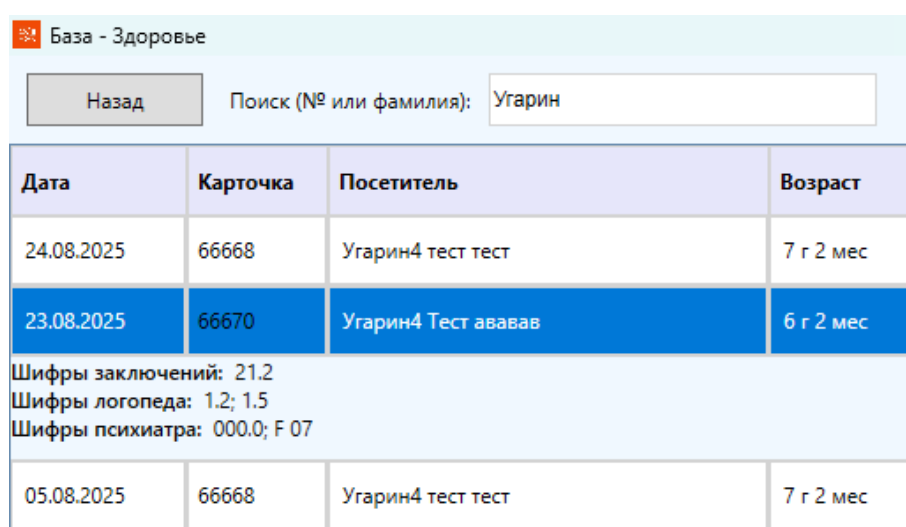
Также, как и предыдущие окна, данное разделено на три строки: панель управления, индикатор загрузки и основная таблица с данными. Таблица отображается с помощью метода «DataGrid», вместо «ListView», как в картотеке. Это обусловлено необходимостью отображения данных из нескольких связанных таблиц и поддержки детализации строк «RowDetails» (при нажатии на прием пациента будет открываться дополнительная информация о его посещении). Дополнительно для обеспечения большей производительности была включена виртуализация с параметром «Recycling» - это означает, что контейнеры элементов повторно используются вместо создания новых при прокрутке, а также включена прокрутка не по элементам, а по пикселям, что сделает ее более плавной.

Структура колонок таблицы:

Заголовок	Привязка	Особенности
Дата	Дата_обращения	Форматирование dd.M M.yyyu
Карточка	C1_Карточки_пациентов.Внутр_номер	Шаблонная колонка с обработчиком клика
Посетитель	Фамилия + Имя + Отчество	MultiBinding для составного ФИО
Возраст	Дата_рождения	Конвертер Age
Возр.группа	Возрастные_группы_детей.Возрастная_группа	Навигационное свойство
ОУ	Список_ОУ.Наименование	Образовательное учреждение

Причина обращения	Причины_Обращения.Причины_обращения1	Справочная таблица
Специалист	ФИО_специалиста + Специализация	Составное поле через MultiBinding

Также реализована выдвигаемая область с дополнительной информацией, это означает что пользователь может развернуть любую строку для просмотра медицинских кодов (шифров) без открытия полной карточки пациента.



База - Здоровье

Назад Поиск (№ или фамилия): Угарин

Дата	Карточка	Посетитель	Возраст
24.08.2025	66668	Угарин4 тест тест	7 г 2 мес
23.08.2025	66670	Угарин4 Тест ававав	6 г 2 мес
Шифры заключений: 21.2 Шифры логопеда: 1.2; 1.5 Шифры психиатра: 000.0; F 07			
05.08.2025	66668	Угарин4 тест тест	7 г 2 мес

Рисунок 18 — Пример открытой строки журнала

Выделю особенность открытия журнала — при нажатии кнопки «Журнал» главное меню не закрывается, позволяя пользователю пользоваться остальным функционалом приложения, держа журнал открытым. Данное действие позволяет прогрузить в памяти журнал только один раз и излишне не нагружать систему. Также добавлен метод `CreateDatabaseIndexes()` — он создаёт индексы в базе данных для ускорения запросов и метод `LoadInitialDataAsync()` — загружает данные постепенно, не пытаясь открыть всё разом. Обращение к базе данных происходит через прямые SQL запросы, обходя Entity Framework и давая прирост производительности при загрузке первых записей.

2.7 Новая карточка

Центральным элементом разработанного приложения является карточка пациента – «NewCard». Данное окно дает полный набор функций для создания новой карточки, просмотра и редактирования существующей, а также содержит дополнительные элементы для занесения дополнительной информации о пациенте.

Также, как и остальные окна, данное имеет идентичные общие настройки (цвет, шапка, шрифт, иконка и т.д.). Интерфейс разделен на четыре строки, а каждая строка имеет разделения от одного до четырех столбцов, в зависимости от планируемого наполнения. Обязательные поля для заполнения выделены символом «*». Поле «Возраст» реализовано как текстовое поле с автоматическим вычислением на основе выбранной даты рождения. Обновление происходит в обработчике события «LayoutUpdated», которое срабатывает при любом изменении окна — в частности, при изменении значения в элементе «DatePicker» (календарь). Вычисление возраста выполняется с точностью до месяцев. Блок «Адрес» включает поле для ввода текста и флажок «Др. Район» (другой район), который позволяет отслеживать пациентов, прикрепленных к учреждению не по месту жительства. Для родителей предусмотрены поля: возрастная группа и уровень образования. Оба поля реализованы через элементы «ComboBox» (выпадающий список) с возможностью ручного ввода. Возрастные группы родителей загружаются из справочной таблицы «Возрастные_группы_родителей», аналогично загружается уровень образования. Семейный статус фиксируется с помощью флажков «CheckBox», охватывающих следующие категории: неполная семья, многодетная семья, приёмная семья, асоциальная семья, мигранты, группа риска. Данная классификация позволяет формировать статистику и корректировать методы работы с пациентом. Все флажки имеют тип «bool» с допустимым значением «null», на случай неопределенности со статусом.

Рисунок 19 — Карточка пациента

Окно «NewCard» реализует два сценария использования. Первый — окно принимает таблицу «C1_Карточки_пациентов» и используется для редактирования существующей карточки. Второй — окно принимает номер карточки и выполняет самостоятельное извлечение данных из базы. Данная особенность позволяет вызывать окно из различных частей системы — из окна картотеки (где объект уже загружен), из окна журнала (где доступен только номер) или при создании нового приема, о чем будет написано позже. Также хочу уточнить, что метод «InitializeComboBoxes()» заполняет все выпадающие списки данными из справочных таблиц. Вынос этой логики в отдельный метод обеспечивает повторное использование кода и упрощает его использование.

2.8 Новый прием

Следующим функциональным модулем является окно «Новый приём» (NewJoin), предназначенное для записи обращения пациента в организацию. Данное окно позволяет создавать новую запись о посещении и редактировать уже существующую информацию о приёме. Интерфейс окна содержит десять строк, что обеспечивает логическую группировку элементов

управления по смысловым блокам. Верхняя панель содержит дату приёма и набор флажков, определяющих форму оказанной помощи. В ней располагается «DatePicker» — элемент выбора даты приёма (интерактивный календарь), значение по умолчанию устанавливается на текущую дату при инициализации окна, флажки «CheckBox» — первичный приём, консультация, диагностика или коррекционное занятие. Слева в верхней панели размещена кнопка «Назад» (Back), возвращающая пользователя в главное меню приложения.

Затем идет блок идентификации пациента, с самым важным элементом — полем поиска пациента «ComboBox», обладающее следующими особенностями реализации:

- «IsEditable="True"» позволяет пользователю вводить фамилию вручную для быстрого поиска.
- При вводе текста в поле запускается механизм автодополнения: поиск инициируется при вводе минимум двух символов, при этом используется отложенный вызов данных с задержкой 300 миллисекунд, что предотвращает чрезмерную нагрузку на сервер при интенсивном наборе текста, так как приемов может быть десятки тысяч.
- Каждый элемент списка формируется таким образом: «Фамилия Имя Отчество (дата рождения)», что позволяет однозначно идентифицировать пациента без необходимости открывать его полную карточку.
- После выбора пациента из списка автоматически заполняются поля «Номер карточки» (cartnum) и «Возраст на приёме» (ageatmoment). Номер извлекается из базы данных на основе выбранного ФИО, а возраст рассчитывается с точностью до месяцев на текущую дату.
- Кнопка «Открыть карточку» (OpenCard) становится активной и обеспечивает быстрый переход к детальному просмотру и редактированию карточки выбранного пациента. При нажатии передаётся номер карточки в

конструктор окна «NewCard2», после чего новое окно отображается поверх текущего. Об окне NewCard2» я расскажу чуть позже.

Также имеются поля, относящиеся к возрастной группе пациента и учёту присутствующих на приёме лиц. Имеется поле «Причина обращения» (reasonapp) — выпадающий список, наполняемый данными из справочной таблицы «Причины_Обращения». Также есть возможность внести сведения о специалисте, проводившем приём.

Наиболее информативной является строка, в которой размещён блок из трёх списков для выбора медицинских шифров и кодов заключений. Каждый список обеспечивает множественный выбор (SelectionMode="Multiple") и снабжён уникальным шаблоном отображения «DataTemplate», выводящим код шифра и его текстовое наименование в двух колонках. К трём спискам относятся:

- Шифры заключений (listBox) — итоговые коды по результатам комплексной диагностики;
- Шифры логопеда (listBox1) — коды, отражающие логопедические заключения;
- Шифры психиатра (listBox2) — коды, отражающие заключения врача-психиатра.

Под каждым списком размещено текстовое поле (List1, List2, List3), в котором отображаются шифры всех выбранных элементов через точку с запятой. Данное решение улучшает наглядность и позволяет при сохранении записать в таблицу «C2_Прием_пациентов» строковое представление выбранных кодов. Обновление текстовых полей происходит при любом изменении выделения в любом из списков через единый обработчик «listBox_SelectedIndexChanged». Дополнительно имеется возможность внести информацию об источнике направления пациента в организацию, образовательном учреждении и программе обучения. Данные

поля реализованы в виде выпадающих списков «ComboBox» с возможностью ручного ввода для оперативного добавления новых значений без остановки работы приложения. Последняя строка содержит поле «Дополнительная информация», а справа размещена кнопка «Сохранить в журнале», при нажатии на которую происходит запись сформированного объекта приёма в базу данных.

Рисунок 20 — Окно создания нового приема.

Для обеспечения мгновенного отклика интерфейса, конкретно в блоке поиска пациента, при работе с большой базой данных (более 3000 обращений ежегодно и уже существующей базой с более чем 30000 приемов) в окне «Новый приём» реализован комплекс оптимизационных механизмов.

Первое — подавление мерцания экрана при вводе данных. Поиск запускается не при каждом нажатии клавиши, а через 300 миллисекунд после прекращения ввода. Это предотвращает отправку десятков одинаковых запросов в БД при быстром наборе фамилии.

Второе — асинхронная загрузка. При открытии окна подгружается только 100 первых записей, а не вся таблица целиком. Полноценный поиск по базе активируется лишь при вводе данных в меню поиска.

Третье — кэширование. Список пациентов сохраняется в памяти на 5 минут. При повторном открытии окна или частых поисках данные берутся из кэша, а не из базы данных.

Четвёртое — индексация. При запуске приложения проверяется наличие индекса в SQL Server по полям «Фамилия» и «Имя». Индекс ускоряет поиск в десятки раз при больших объёмах данных.

Пятое — контекст без отслеживания (AsNoTracking()). Entity Framework не сохраняет информацию об изменениях объектов при чтении, что снижает потребление оперативной памяти и ускоряет выборку.

В совокупности эти меры позволяют получать результаты поиска практически мгновенно даже при многотысячной базе пациентов, устраняя проблему зависаний, характерную для старой системы на Microsoft Access. Функциональность окна «Новый приём» полностью покрывает потребности регистратуры и специалистов ЦППМСП «Здоровье» в части оперативного внесения информации о каждом обращении пациента.

2.9 Вспомогательное окно

В приложении имеется вспомогательное окно «NewCard2», оно дает возможность просматривать и редактировать информацию о пациенте, а открывается оно при нажатии кнопки открытия карточки в окне «Новый прием». В отличие от окна «NewCard» (которое предназначено для создания новой карточки), «NewCard2» является только просмотрно-редакционным и не содержит полей для создания нового пациента. Оба окна используют общую структуру данных, но решают разные задачи. При открытии данного окна данные пациента извлекаются одним обращением к таблице «С1_Карточки_пациентов». Значения типа (true/false) преобразуются в читаемый вид «Да/Нет» через созданный мною преобразователь «BoolToYesNoConverter».

```

public class BoolToYesNoConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        if (value is bool boolValue)
        {
            return boolValue ? "Да" : "Нет";
        }
        return "Нет";
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        if (value is string stringValue)
        {
            if (string.IsNullOrEmpty(stringValue))
            {
                return null;
            }
            return stringValue == "Да";
        }
        return null;
    }
}

```

Рисунок 21 — Преобразователь булевых значений

Также числовые коды (возраст родителей, уровень образования, русский язык, условия проживания) преобразуются в текстовые описания через отдельные методы, а на основе выбранной даты рождения вычисляется полный возраст пациента. Все поля доступны для изменения, после нажатия кнопки «Сохранить изменения» они фиксируются через метод «SaveChanges()». Также в журнале приема и картотеке были добавлены методы, реагирующие на двойное нажатие на номер карточки, позволяющие открыть ее.

```

private void CardNumber_MouseLeftButtonDown(object sender, MouseButtonEventArgs e)
{
    // Открывать карточку только по двойному клику левой кнопкой
    if (e.ClickCount != 2)
        return;

    if (sender is TextBlock textBlock)
    {
        // Берем номер карточки из элемента данных (надежнее, чем парсить текст)
        if (textBlock.DataContext is Entities.C1_Карточки_пациентов patient)
        {
            if (!patient.Внутр_номер.HasValue)
                return;

            int cardNumber = patient.Внутр_номер.Value;
            try
            {
                var window = new NewCard2(cardNumber);
                window.Owner = this;
                window.Show();
                window.Activate();
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Не удалось открыть карточку {cardNumber}: {ex.Message}", "Ошибка",
                                MessageBoxButton.OK);
            }
        }
    }
}

```

Рисунок 22 — Обработчик открытия карточки в картотеке

2.10 Создание отчетов

По требованиям руководства ЦППМСП «Здоровье» была добавлена возможность создавать несколько различных отчетов, для дальнейшего анализа и печати: печать журнала, аналитический отчет, статистика обращений, статистика обращений, статистика по специалистам, статистика по заведениям и статистика по возрастам.

Первый добавленный элемент был - печать журнала. Его главной задачей является создание PDF документа журнала приема за выбранные даты с уже открывшимися шифрами специалистов, которые, в самом журнале, надо было открывать нажатием на прием. Для этого в главном меню во второй строке появился пункт с кнопкой «Печать журнала» и двумя календарными элементами «DatePicker».

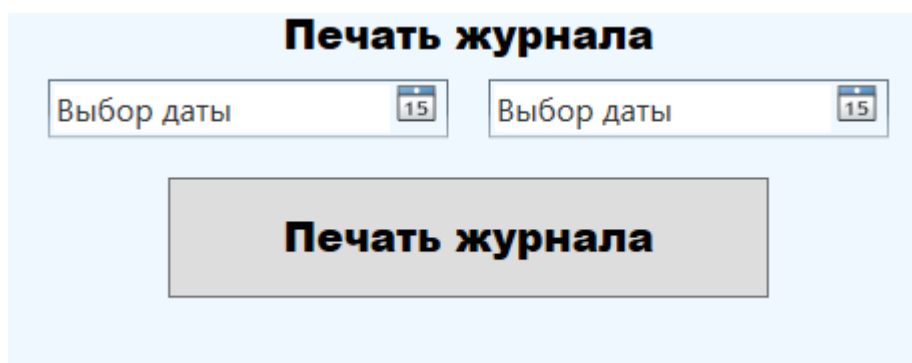


Рисунок 23 — Печать журнала

Приступив к написанию кода, для обеспечения единообразия и сокращения его дублирования при создании всех отчетных форм был разработан базовый класс «Rep_base». Данный класс включает общую функциональность, необходимую каждому конкретному генератору, и задает единый стиль для всех отчетов. Класс «Rep_base» содержит четыре ключевых компонента:

1. Строка подключения к базе данных. Вынесение строки подключения в базовый класс позволяет изменить параметры соединения централизованно,

не затрагивая код каждого отдельного генератора отчетов, если это необходимо.

2. Метод `CreateBaseFont()` обеспечивает загрузку стандартного кириллического шрифта для корректного отображения русскоязычного текста в PDF-документах. Также используется кодировка, позволяющая отображать символы кириллицы без искажений.

3. Метод `CreateFilePath(string fileName)` определяет директорию для сохранения сгенерированных отчётов. Все PDF-файлы помещаются в папку «Мои документы» текущего пользователя ОС «Windows».

4. Метод `ValidateDateRange(DateTime? dateStart, DateTime? dateEnd)` выполняет проверку, что обе даты указаны пользователем (не равны null), что дата начала не превышает дату окончания. При нарушении любого из условий метод генерирует исключение «`ArgumentException`» с понятным текстовым описанием ошибки.

5. Метод `FormatDateRange(DateTime dateStart, DateTime dateEnd)` позволяет визуально понятно отображать выбранный период времени в заголовках отчётов. Возвращает строку в формате «ДД.ММ.ГГГГ — ДД.ММ.ГГГГ».

Каждый конкретный генератор отчёта наследует класс «`Rep_base`». Благодаря этому разработка нового типа отчёта сводится к реализации единственной логики извлечения и представления данных, а в это же время подключение к БД, инициализация шрифтов, формирование пути к файлу и отображение дат уже готовы к использованию.

Затем, используя данный класс, был создан «`Rep_journal.cs`». Как было описано ранее, данный класс реализовывает печать журнала пациентов с уже открытыми шифрами заключений, ниже можно наблюдать созданный мною пример печати.

Журнал приема пациентов за период с 03.07.2025 по 01.08.2025

Дата	Карточка	Посетитель	Возраст	Возр.группа	ОУ	Специалист
03.07.2025	22222	Угарин Никита Александрович	1	ДОО	Школа №99 Старт	Г.
Шифры заключений: 21.5			Шифры логопеда: 1.1; 1.3		Шифры психиатра: F 06.6; F 06.7; F 07.0	

Рисунок 24 — Сгенерированная печать журнала

Аналогичным образом, в главном меню, в третьей колонке, были созданы навигационные кнопки для создания аналитических отчетов. Данный блок также имеет два объекта «Datepicker» для выбора дат и пять различных кнопок для генерации отчетов.

Рисунок 25 — Кнопки для создания отчетов

Краткая логика работы всех отчетов:

1. «Rep_analytical» (Аналитический отчет). Выполняет четыре SQL-запроса для подсчёта обращений по причинам отдельно для детей, родителей, педагогов и в целом. Формирует PDF с четырьмя аналитическими таблицами и справочником причин. Файл сохраняется с именем Аналитический_Отчет_датавремя.pdf.

2. «Rep_statistic» (Статистика обращений). Выполняет шесть последовательных запросов для подсчёта уникальных пациентов, общего числа посещений, первичных приёмов, консультаций, диагностик и занятий. Формирует PDF с портретной ориентацией, выводя статистику в виде маркированного списка. Файл сохраняется как Статистика_обращений_датавремя.pdf.

3. «Rep_specialist» (Статистика по специалистам). Выполняет запрос с группировкой по каждому специалисту, подсчитывая количество уникальных пациентов, которые к нему обращались. Формирует таблицу из трёх колонок (ФИО, специализация, количество пациентов) с итоговой строкой суммы. Файл сохраняется как Статистика_специалистов_датавремя.pdf.

4. «Rep_educations» (Статистика по учебным заведениям). Выполняет запрос с соединением таблицы учреждений и приёмов, подсчитывая уникальных пациентов из каждого ОУ. Формирует двухколоночную таблицу (название учреждения, количество пациентов) с итогом. Файл сохраняется как Статистика_по_заведениям_датавремя.pdf.

5. «Rep_AgeDistribution» (Статистика по возрастам). Выполняет сложный запрос с CTE, вычисляя точный возраст пациентов на момент приёма и подсчитывая уникальных детей в диапазоне 0–18 лет. Исключает дублирование одного пациента в пределах 10-дневного интервала. Файл сохраняется как Статистика_по_возрастам_датавремя.pdf.

2.11 Сборка приложения

Для того чтобы пользователи, не имеющие установленной среды разработки Visual Studio, могли использовать разработанное приложение, его необходимо опубликовать в виде исполняемого файла с набором всех необходимых библиотек. Для корректной сборки необходимо на панели инструментов выбрать переключатель конфигурации сборки, затем выбрать значение «Release» вместо «Debug». В режиме «Release» компилятор

выполняет оптимизацию кода, удаляет отладочную информацию и уменьшает итоговый размер выходных файлов. Также следует убедиться, что в выпадающем списке платформы выбрано значение «Any CPU», что обеспечит совместимость приложения с различными архитектурами процессоров.

После выбора конфигурации Release необходимо нажать правой кнопкой мыши на имени проекта в обозревателе решений и выбрать пункт контекстного меню «Собрать» (Build). Visual Studio начнёт компиляцию проекта, отображая ход процесса в окне вывода. При успешной сборке в нижней части экрана появится сообщение «Сборка успешно завершена».

После успешной сборки исполняемые файлы располагаются в папке проекта. В этой папке помимо основного исполняемого файла находятся все зависимые библиотеки (например, `itextsharp.dll`, `EntityFramework.dll` и другие), а также папка «Images» с необходимыми графическими ресурсами (логотипом и иконками).

Для передачи приложения конечному пользователю необходимо скопировать всю папку «Release» на внешний носитель или заархивировать её для дальнейшей отправки. Пользователю достаточно запустить файл `Firamir.exe` — приложение не требует установки и готово к работе после настройки подключения к серверу базы данных. Перед первым запуском необходимо убедиться, что настроено подключение к удалённому серверу.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была достигнута поставленная цель — разработано полнофункциональное приложение для администрирования клиентской базы данных медицинской организации ЦППМСП «Здоровье» Петроградского района Санкт-Петербурга. Реализованное программное решение полностью заменило устаревшую систему на базе Microsoft Access.

В первой главе работы был проведён детальный анализ существующей базы данных, выявлены её недостатки и сформулированы требования к новому приложению. Рассмотрены различные методологии разработки и инструменты для миграции данных, после чего обоснован выбор технологического стека: «MS SQL Server», платформа «.NET Framework» и «WPF» для реализации графического интерфейса, язык программирования C#. Инструмент миграции баз данных «SSMA».

Во второй главе детально описана практическая реализация всех функциональных модулей приложения. Особое внимание уделено модулю формирования печатных отчётов в формате PDF. Разработаны шесть видов отчётов, что является отличительной чертой данного приложения, от Microsoft Access.

Все задачи, поставленные во введении, выполнены в полном объёме. Разработанное приложение имеет понятный интерфейс, высокую скорость работы, устойчивость к большим объёмам данных и обеспечивает централизованное хранение информации о пациентах, оперативный поиск, регистрацию приёмов и формирование статистической отчётности. Внедрение данного программного продукта позволит сотрудникам ЦППМСП «Здоровье» оптимизировать процесс учёта пациентов, сократить время на поиск и обработку информации, исключить ошибки, связанные с ручным вводом данных, и получать актуальные аналитические сводки.

СПИСОК ЛИТЕРАТУРЫ

1. Как соединить SQL с Visual Studio. Соединение Visual Studio и SQL Server. Пошаговое руководство. — Текст : электронный // graph.org : [сайт]. — URL: <https://graph.org/Kak-soedinit-SQL-s-Visual-Studio-Soedinenie-Visual-Studio-i-SQL-Server-Poshagovoe-rukovodstvo-12-30> (дата обращения: 11.05.2026).
2. C# и WPF | Взаимодействие с базой данных. — Текст : электронный // metanit.com : [сайт]. — URL: <https://metanit.com/sharp/wpf/19.1.php> (дата обращения: 11.05.2026).
3. Создание простого приложения данных с помощью WPF и Entity Framework 6. — Текст : электронный // learn.microsoft.com : [сайт]. — URL: <https://learn.microsoft.com/ru-ru/visualstudio/data-tools/create-a-simple-data-application-with-wpf-and-entity-framework-6?view=visualstudio> (дата обращения: 11.05.2026).
4. Лучшие компиляторы кода и IDE для C++. — Текст : электронный // Skillbox Media : [сайт]. — URL: <https://skillbox.ru/media/code/luchshie-kompilyatory-koda-i-ide-dlya-c/> (дата обращения: 11.05.2026).
5. XAML in WPF. — Текст : электронный // Microsoft Learn : [сайт]. — URL: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/xaml/> (дата обращения: 11.05.2026).
6. Перенос данных из Access в SQL Server. — Текст : электронный // Microsoft Learn : [сайт]. — URL: <https://learn.microsoft.com/ru-ru/sql/sql-server/migrate/guides/access-to-sql-server?view=sql-server-ver17> (дата обращения: 11.05.2026).
7. Open Specifications: Windows Protocols. — Текст : электронный // Microsoft Learn : [сайт]. — URL: https://learn.microsoft.com/en-us/openspecs/windows_protocols/mc-edmx/688b06d4-aa08-4b58-b200-f286ce696383 (дата обращения: 11.05.2026).

8. Working with DbContext. — Текст : электронный // Microsoft Learn : [сайт]. — URL: <https://learn.microsoft.com/ru-ru/ef/ef6/fundamentals/working-with-dbcontext> (дата обращения: 11.05.2026).
9. C# и WPF | Первое приложение в Visual Studio. — Текст : электронный // metanit.com : [сайт]. — URL: <https://metanit.com/sharp/wpf/1.2.php> (дата обращения: 11.05.2026).
10. Обзор C# библиотек для работы с PDF / Хабр. — Текст : электронный // habr.com : [сайт]. — URL: <https://habr.com/ru/articles/112707/> (дата обращения: 11.05.2026).
11. Создание приложения WPF. — Текст : электронный // Microsoft Learn : [сайт]. — URL: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/app-development/building-a-wpf-application-wpf> (дата обращения: 11.05.2026).